

Métricas de calidad – Greenet

A continuación, se presentan 4 métricas de calidad del proyecto a la fecha, 2 que se encuentran implementadas dentro del código base de Greenet y 2 tomadas directamente desde el análisis realizado por la plataforma SonarCloud

- Métricas implementadas en el código:
Se implementaron a través de una clase que contiene su propio main y permite visualizar las métricas:

```
METRICAS DE CODIGO
=====
LoginController.java:
  Complejidad: 15 (ALTA)
  Lineas: 159 (LARGO)

SignupController.java:
  Complejidad: 28 (MUY ALTA)
  Lineas: 247 (LARGO)

PublicacionHogar.java:
  Complejidad: 1 (BAJA)
  Lineas: 33 (CORTO)
```

- Complejidad Ciclomática: En los resultados generados por la función se evidencia que algunas clases presentan niveles elevados de complejidad ciclomática, especialmente en los controladores. Esta métrica mide el número de caminos lógicos posibles dentro del código, y un valor alto indica que la clase tiene demasiadas decisiones, bifurcaciones o condiciones anidadas, lo que aumenta el riesgo de errores y dificulta su mantenimiento.

La complejidad ciclomática es una métrica fundamental porque permite anticipar la propensión del código a fallos y su dificultad de comprensión.

En proyectos de mayor escala, mantener este valor bajo es clave para garantizar que el sistema crezca de manera sostenible.

Como acción correctiva, se recomienda replantear los controladores más complejos, separando las validaciones, la lógica de negocio y las operaciones con base de datos en componentes independientes. De esta manera, se mejora la legibilidad, la mantenibilidad y se reduce la probabilidad de errores lógicos.

- Cantidad de líneas por función: muestra que varios archivos, tienen una longitud de código exageradamente amplia en un solo archivo o método indica que no se creó de forma granular, ya que varios procesos se están ejecutando dentro de la misma función en lugar de dividirse en responsabilidades más pequeñas

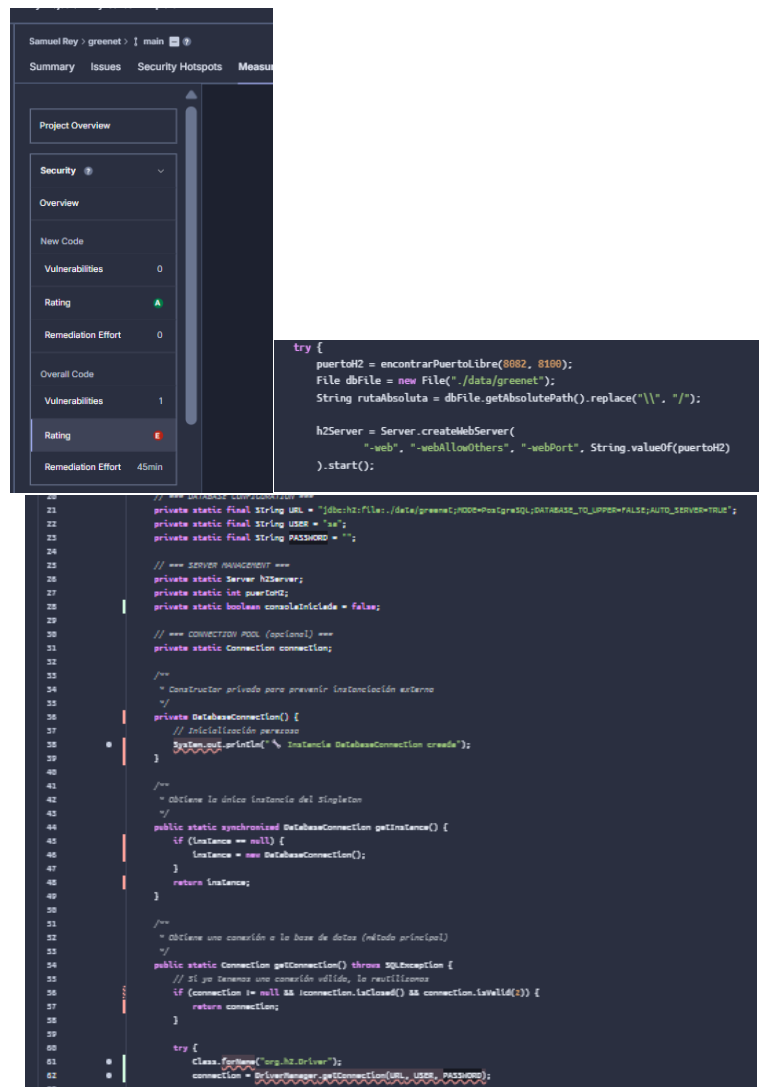
El tamaño no solo complica la lectura y comprensión del código, sino que también aumenta la posibilidad de cometer errores. En cambio, la clase más

pequeña, en este ejemplo: PublicacionHogar.java (33 líneas), refleja una buena segmentación y diseño.

En términos generales, esta métrica es un componente de la mantenibilidad del código y su claridad

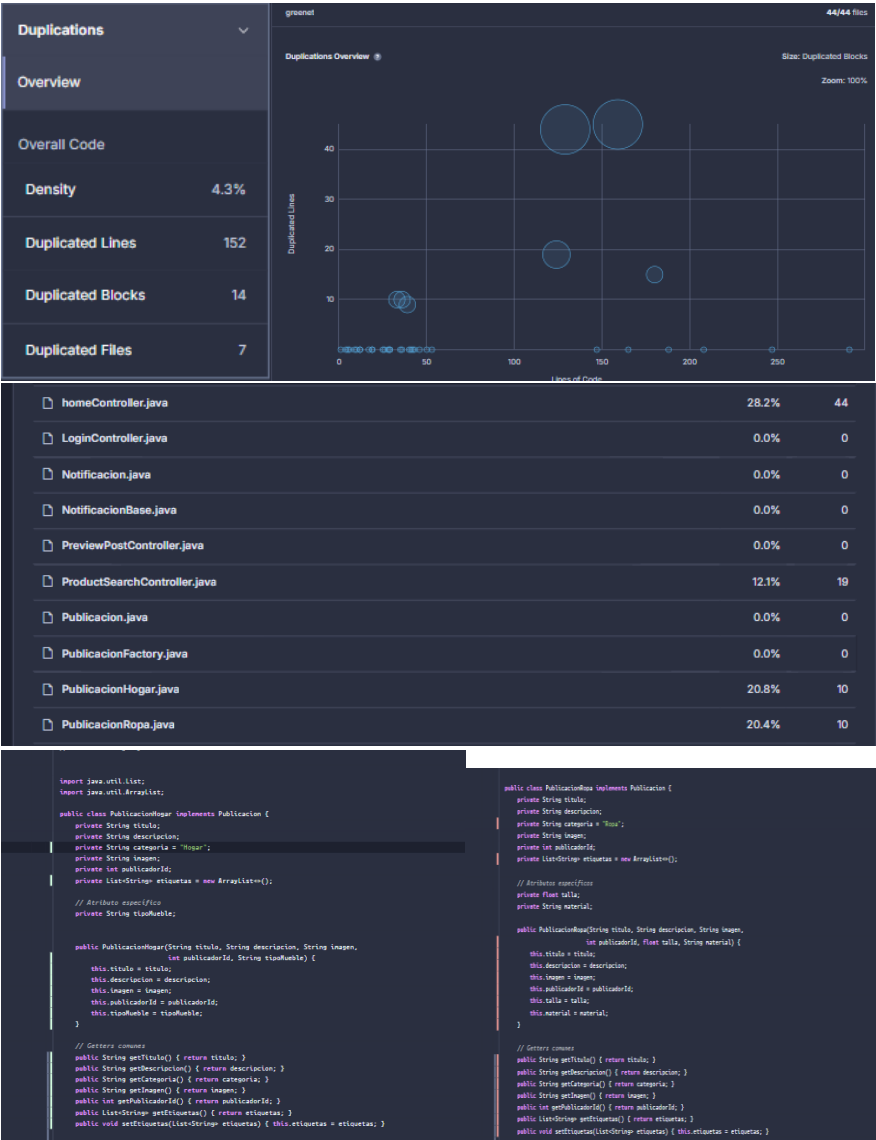
Para mejorar este aspecto, se sugiere dividir los métodos más largos en métodos más pequeños específicos, que sean usados luego en la clase. Con esto, el código se vuelve más comprensible y mantenible.

- Métricas de SonarCloud
 - Vulnerabilidad: En el reporte proporcionado por SonarCloud se detecta un problema grave en la clase DataBaseConnection.java. Dicha clase contiene los elementos necesarios para realizar la conexión a la base de datos embebida utilizando la herramienta H2. A continuación se presentan pantallazos del análisis de SonarCloud:



En la clase se evidencia que para realizar la inicialización de la base de datos embebida, no se le tiene asignada la contraseña, por lo cual cualquier persona podría ingresar a la base de datos obteniendo la información (URL y usuario) y modificar alguna información. También se detecta que la base de datos está disponible para cualquier red externa, lo cual de igual forma pone en juego la integridad de los datos. Debe hacerse uso de las configuraciones de secretos de github para asignar una contraseña y protegerla. Por otra parte, se debe realizar un ajuste a la configuración de la base de datos para protegerla de accesos externos

- Duplicaciones: En el reporte generado por SonarCube se detectó un cierto porcentaje de líneas duplicadas en el código, a continuación se muestra el análisis con datos exactos



```

public class PublicacionTecnologia implements Publicacion {
    private String titulo;
    private String descripcion;
    private String categoria = "Tecnologia";
    private String imagen;
    private int publicadorId;
    private List<String> etiquetas = new ArrayList<>();

    // Atributos especificos
    private String modelo;
    private String marca;
    private boolean garantia;

    public PublicacionTecnologia(String titulo, String descripcion, String imagen,
                                  int publicadorId, String modelo, String marca, boolean garantia) {
        this.titulo = titulo;
        this.descripcion = descripcion;
        this.imagen = imagen;
        this.publicadorId = publicadorId;
        this.modelo = modelo;
        this.marca = marca;
        this.garantia = garantia;
    }

    // Getters comunes
    public String getTitulo() { return titulo; }
    public String getDescripcion() { return descripcion; }
    public String getCategoria() { return categoria; }
    public String getImagen() { return imagen; }
    public int getPublicadorId() { return publicadorId; }
    public List<String> getEtiquetas() { return etiquetas; }
    public void setEtiquetas(List<String> etiquetas) { this.etiquetas = etiquetas; }
}

```

La duplicación en este caso está indicando un error de planteamiento en el modelado del software, debido a que la clase base publicación se encuentra como interfaz. Generando que en sus implementaciones se repita el código de los getters comunes, como se aprecia en las imágenes. En este caso nos está indicando que se debe manejar una clase abstracta, para reducir el código duplicado, facilitando así la mantenibilidad del código y mejorando la coherencia respecto al dominio del problema. A nivel general, podemos decir que la métrica de duplicaciones puede ayudar a la identificación de problemas de mantenibilidad del código e incluso errores de diseño de clases